颜色反应

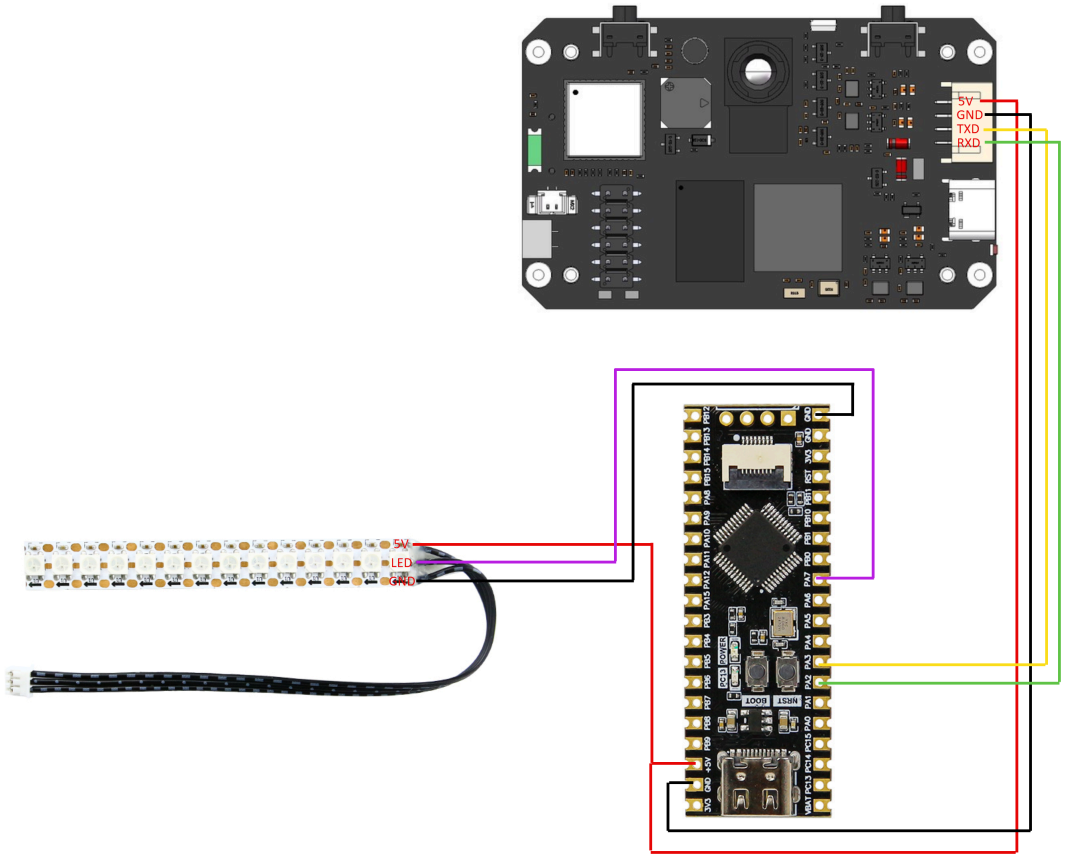
1.开篇说明

本案例使用的是亚博14灯珠 RGB灯条模块，没有使用到电机，RGB灯条不在套餐内，需要自行另外购买。

2.实验准备

K230视觉模块、亚博STM32F103C8T6核心板、14灯珠 RGB灯条。

硬件接线



接线引脚

14灯珠 RGB灯条	亚博STM32F103C8T6
5V	5V
GND	GND
LED	PA7

K230	亚博STM32F103C8T6
5V	5V
GND	GND
TXD	PA3
RXD	PA2

3.实验操作

1.按照教程的硬件接线说明进行接线。

2.给STM32烧录ProtocolHub工程

3.将Color_Reaction.py放置于K230的sdcard根目录下，并重命名为main.py，然后reset或者重新上电。

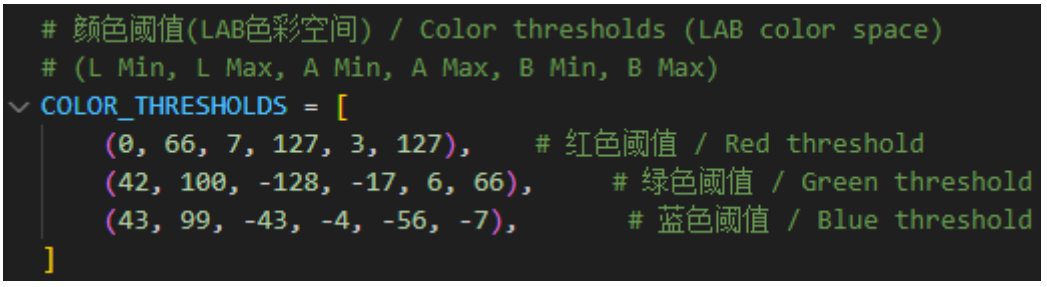
如果不清楚K230如何烧入程序，请先看K230的快速入门章节教程。

4.掰动K230模块支架到合适的角度，接上电源，放一个色彩比较鲜艳的物体在K230相机前面。

5.当K230初始化成功后，会开始识别设定好的颜色，同时RGB灯条也会变成对应的颜色。

注意：

Color_Reaction.py中提供红、绿、蓝，三种预设颜色的识别，如果识别效果不好，可以看K230的颜色识别教程，学习如何将LAB阈值更换为适配自己环境的阈值。



```
# 颜色阈值(LAB色彩空间) / Color thresholds (LAB color space)
# (L Min, L Max, A Min, A Max, B Min, B Max)
COLOR_THRESHOLDS = [
    (0, 66, 7, 127, 3, 127),     # 红色阈值 / Red threshold
    (42, 100, -128, -17, 6, 66),     # 绿色阈值 / Green threshold
    (43, 99, -43, -4, -56, -7),       # 蓝色阈值 / Blue threshold
]
```

4.代码解析

- main.c

```
#define MOTOR_TYPE 5    //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520电机
                        //1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC reduction motor 5:L type 520 motor

//修改当前要运行的颜色案例  Modify the color case currently being run
ColorMode color_mode = 1;  //1:巡线、2:跟踪  1: Patrol, 2: Tracking

//选择是否在初始化完成后，小车自主向前走，0为否，1为是(自主避让案例使用)
//Select whether the car moves forward autonomously after initialization is completed, 0 for no, 1 for yes (used in autonomous avoidance cases)
int Car_Auto_Drive = 0;

int main(void)
{
```

```
    //硬件初始化 Hardware Initialization
    BSP_init();
    Set_Motor(MOTOR_TYPE);

    if(Car_Auto_Drive)//K230上电初始化需要时间，等待初始化完成之后再开始前进。    It takes
time for K230 to power on and initialize. Wait until the initialization is
complete before moving forward.
    {

delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_
ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);
    }
    printf("Init Succeed\r\n");
    while(1)
    {
        Pto_Loop();
        delay_ms(10);
    }
}
```

MOTOR_TYPE：用于设置使用的电机类型，根据自己现使用的电机对照着注释修改对应的数字。

color_mode：要运行视觉巡线案例的时候使用1，如果是颜色跟踪案例的时候使用2

Car_Auto_Drive：控制小车在初始化后自动前进，默认为0，不自动前进。只有使用自主避让案例的时候需要开启

BSP_init：硬件初始化

Set_Motor：与四路电机驱动板通信，设置电机相关参数

Pto_Loop：持续处理接收到的K230的消息。

- yb_protocol.c

```
/**
 * @Brief: 数据分析 Data analysis
 * @Note:
 * @Parm: 传入接受到的一个数据帧和长度    Pass in a received data frame and length
 * @Retval:
 */
void Pto_Data_Parse(uint8_t *data_buf, uint8_t num)
{
    uint8_t pto_head = data_buf[0];
    uint8_t pto_tail = data_buf[num-1];

    //校验头尾  Check head and tail
    if (!(pto_head == PTO_HEAD && pto_tail == PTO_TAIL))
    {
        //DEBUG_PRINT("pto error:pto_head=0x%02x , pto_tail=0x%02x\n", pto_head,
pto_tail);
        return;
    }

    uint8_t data_index = 1;
```

```
    uint8_t field_index[PTO_BUF_LEN_MAX] = {0};
    int i = 0;
    int values[PTO_BUF_LEN_MAX] = {0};
    char msgs[][PTO_BUF_LEN_MAX] = {0};

    //分割字段  Split Field
    for (i = 1; i < num-1; i++)
    {
        if (data_buf[i] == ',')
        {
            data_buf[i] = 0;
            field_index[data_index] = i;
            data_index++;
        }
    }

    //解析长度与功能ID  Parsing length and function ID
    for (i = 0; i < 2; i++)
    {
        values[i] = Pto_Char_To_Int((char*)data_buf+field_index[i]+1);
    }

    uint8_t pto_len = values[0];
    uint8_t pto_id = values[1];


 ParseCommonFields(pto_id,data_buf,pto_len,field_index,data_index,values,msgs);
}

//获取数据函数  Get data function
void ParseCommonFields(
    int func_index,       // 功能索引（对应func_table中的位置） Function index
(corresponding to the position in func_table)
    uint8_t *data_buf,    // 原始数据缓冲区 Raw data buffer
    uint8_t num,          // 数据总长度（含头尾）   Total length of data (including
head and tail)
    uint8_t *field_index, // 字段分隔符位置数组（逗号的位置）   Array of field
separator positions (positions of commas)
    uint8_t data_index,   // 总字段数（如数据有6个字段）    Total number of fields
(if the data has 6 fields)
    int* values,          // 存储整数字段的数组 Array to store integer fields
    char msgs[][PTO_BUF_LEN_MAX]        // 存储字符串的指针数组 Array of pointers to
store strings
){
    const FuncDesc* desc = &func_table[func_index-1];

    if(func_index != desc->id)
    {
        DEBUG_PRINT("Unsupported protocol ID: %d, Chosen ID:%d\n",
func_index,desc->id);
        return;
    }

    for (int i = 0; i < desc->field_count; i++)
    {
        const FieldMeta* meta = &desc->fields[i];
```

```
        uint8_t pos = meta->pos;

        if (meta->type == FIELD_STRING) {
            int start = field_index[pos] + 1;
            int end = (pos+1 < data_index) ? field_index[pos+1] : (num-1);
            int len = end - start;

            len = (len < PTO_BUF_LEN_MAX-1) ? len : PTO_BUF_LEN_MAX-1;
            memcpy(msgs[i], data_buf+start, len);
            msgs[i][len] = '\0';
        } else {
            values[pos] = Pto_Char_To_Int((char*)data_buf + field_index[pos] +1);
        }
    }
    desc->handler(values, msgs);
}
```

将接收到的符合协议的数据进行解析，获取数据中的功能ID号、携带的数据信息，并执行对应功能的处理函数。本工程使用元数据解析的方法。核心思想是将协议格式的描述（字段位置、类型等）与解析逻辑分离，通过数据结构（如结构体数组）定义协议格式，解析函数根据这些描述自动处理数据。

协议中含有不同类型的数据也可以解析出来，想了解实现细节可以自行研究，不作过多的解答。此函数已适配了本店所有的K230通讯协议，不用完全理解也可以正常使用。

- app_motor.c

```
void Color_Rec(const char* msg)
{
    if(strcmp(msg,"RED")==0)
    {
        DEBUG_PRINT("RED!\r\n");
        RGB_Set_Color(255,255,0,0);
        RGB_Update();
        delay_ms(1000);
    }
    else if(strcmp(msg,"GREEN")==0)
    {
        DEBUG_PRINT("GREEN!\r\n");
        RGB_Set_Color(255,0,255,0);
        RGB_Update();
        delay_ms(1000);
    }
    else if (strcmp(msg, "BLUE")==0)
    {
        DEBUG_PRINT("BLUE!\r\n");
        RGB_Set_Color(255,0,0,255);
        RGB_Update();
        delay_ms(1000);
    }
    else {
        DEBUG_PRINT("Unknown Color: %s\n", msg);
    }
```

```
}
```

Color_Rec：收到来自K230的数据后，查看字符串内容是否是红、绿或蓝，如果是的话就设置RGB灯条的颜色。

5.实验现象

打开电源开关，等待系统初始化完成后，屏幕会显示摄像头画面，并且会框起识别到的颜色，如果是红、绿、蓝三种中的其中一种，那么RGB灯条就会亮对应的颜色。

